
Generación de código fuente de la lógica de negocio

Registro de decisión de arquitectura

Documento **SLCP-ADR-0002** – Versión **1.0**

Estado: **ACCEPTED** – decisión de la parte interesada

Reemplaza a: **SLCP-ADR-0001** (estado **SUPERSEDED**)

6 de agosto de 2026

La generación del código fuente de la lógica de negocio a partir de la especificación de requisitos es un requisito del producto, establecido por la parte interesada. Este registro deja constancia de esa decisión, sustituye la restricción de alcance propuesta en SLCP-ADR-0001, enumera las consecuencias que la decisión acarrea sobre los requisitos ya acordados, y define la arquitectura de verificación que hace la decisión defendible ante una auditoría. El documento no discute la decisión: la ejecuta.

Índice

1	Decisión	2
2	Consecuencias asumidas	2
3	Arquitectura de generación verificable	2
3.1	Principio rector	2
3.2	Mecanismos	3
3.3	Estrategia por nivel	3
4	Riesgos residuales	4
5	Cambios aplicados a SLCP-DOC-001	5

1 Decisión

D-02(d) — adoptada. La plataforma SLCP debe generar, a partir de la especificación de requisitos, la totalidad de los artefactos del ciclo de vida, incluido el código fuente de la lógica de negocio. La generación de la lógica de negocio no es una capacidad opcional ni diferible: es un requisito funcional del producto.

SLCP-ADR-0001 proponía excluir del alcance la lógica algorítmica no especificable de forma declarativa. Esa propuesta queda sin efecto. Se conserva de ella únicamente la taxonomía de niveles L0 a L4, que sigue siendo útil no como criterio de exclusión sino como criterio de *estrategia de generación y de verificación*: cada nivel se genera por un camino distinto y se verifica con un rigor distinto.

2 Consecuencias asumidas

Una decisión de alcance de esta naturaleza tiene consecuencias técnicas que deben quedar registradas, no para reabrir el debate sino para que el diseño las contemple desde el inicio en lugar de tropezar con ellas a mitad de camino.

C-01. La reproducibilidad deja de ser universal. La generación de lógica L4 no puede garantizarse determinista. TRC-23 debe reescribirse: en lugar de exigir reproducibilidad, exige *declaración explícita del régimen de determinismo por artefacto*, con registro de los parámetros que permitirían aproximar la reproducción.

C-02. La verificación sustituye a la derivación como garantía. Cuando el artefacto no se deriva por función sino por interpretación, la confianza no puede provenir del procedimiento de generación; debe provenir de la evidencia de verificación. Esto invierte el orden de trabajo: el oráculo de prueba debe existir antes que la implementación.

C-03. La plataforma pasa a ser un sistema de IA en el sentido normativo. Resultan aplicables Int [1] en gestión, registro y supervisión humana, Int [2] como extensión del modelo de calidad, y las guías de Int [3] para la prueba de sistemas basados en IA. Estas tres referencias se incorporan al marco normativo del proyecto.

C-04. La estrategia de reconciliación debe cambiar. La decisión D-03, orientada a una capa generada inmutable, no sobrevive sin ajuste: la lógica de negocio es precisamente donde el desarrollo humano interviene. El ajuste se define en la Sección 4.

C-05. El coste de revisión se traslada al proceso. Revisar lógica generada cuesta aproximadamente lo mismo que escribirla, salvo que la revisión se apoye en evidencia mecánica. Por eso la arquitectura de la Sección 3 no es un añadido de calidad: es la condición que hace viable la decisión.

3 Arquitectura de generación verificable

3.1 Principio rector

La generación de lógica de negocio se apoya en un principio único: **el oráculo antes que la implementación**. La plataforma deriva primero, desde los criterios de aceptación del

requisito, el conjunto de pruebas que definirá la corrección; congela ese conjunto; y solo entonces genera la implementación, iterando hasta que las pruebas pasan. La implementación nunca puede modificar su propio oráculo, porque en ese momento la verificación dejaría de significar nada.

3.2 Mecanismos

M-01. Oráculo derivado y congelado. Los casos de prueba se derivan de los criterios de aceptación aplicando las técnicas de diseño de Int [4], y se sellan con huella criptográfica antes de generar código. Toda alteración posterior del oráculo exige aprobación humana independiente y queda registrada como evento distinto.

M-02. Enriquecimiento obligatorio de la especificación. Si un requisito carece de criterio de aceptación verificable, la plataforma no genera su lógica: solicita la información faltante. Esto refuerza TRC-06 y traslada el esfuerzo al lugar donde sí es productivo, que es la especificación. Un requisito mal escrito no produce código plausible, produce una pregunta.

M-03. Verificación en capas con bloqueo. La promoción de un artefacto L4 exige superar, en orden: compilación limpia; ejecución en verde del oráculo congelado; comprobación de invariantes y propiedades declaradas; medidas automatizadas de Int [5] sin debilidades de seguridad ni de fiabilidad; y cobertura mínima sobre el propio artefacto. El fallo de cualquier capa detiene la promoción.

M-04. Escalado por agotamiento. Si tras un número definido de iteraciones el oráculo no se satisface, o si la complejidad del artefacto supera un umbral configurado, la plataforma detiene la generación y entrega la interfaz, el oráculo y un informe de bloqueo, marcando el elemento para desarrollo humano. Este mecanismo es esencial: convierte el fracaso en un resultado explícito y auditable en lugar de en código plausible pero incorrecto, que es el modo de fallo más peligroso del enfoque adoptado.

M-05. Congelado tras aprobación y regeneración por propuesta. Una vez aprobada por una persona, la implementación queda fijada y versionada. Una regeneración posterior no sobrescribe: produce una propuesta acompañada de la diferencia respecto de la versión vigente y del resultado del oráculo sobre ambas. La sustitución requiere aprobación expresa. Así se resuelve C-04 sin recurrir a regiones protegidas.

M-06. Procedencia ampliada. Además de lo exigido por TRC-21, cada artefacto L4 registra el modelo generador y su versión, los parámetros de muestreo, la instrucción aplicada, el número de iteraciones consumidas hasta alcanzar el verde y la huella del oráculo utilizado.

M-07. Aprobación con constancia de lectura. La aprobación de un artefacto L4 exige constancia explícita por artefacto, no aprobación en bloque. La plataforma debe registrar qué persona revisó qué versión exacta, y debe imponer muestreo obligatorio cuando el volumen haga inviable la revisión exhaustiva.

M-08. Métricas de gobierno de la generación. La plataforma mide y publica la tasa de aceptación en el primer intento, el número medio de iteraciones hasta el verde, la tasa de escalado a desarrollo humano y la densidad de defectos posteriores por artefacto generado. Estas métricas gobiernan la mejora del generador y, si el proyecto tiene finalidad de investigación, constituyen su instrumento de medición principal.

3.3 Estrategia por nivel

Tabla 1: Camino de generación y verificación por nivel

Nivel	Camino de generación	Verificación exigida
L0–L2	Plantillas deterministas sobre el modelo	Compilación y pruebas de contrato; la corrección es estructural
L3	Plantillas deterministas sobre artefactos declarativos de regla	Pruebas generadas que cubren todas las reglas y transiciones; corrección demostrable por construcción
L4	Generación interpretativa contra oráculo congelado, con iteración hasta el verde	M-01 a M-07 en su totalidad; sin evidencia de verificación no hay promoción

El objetivo de diseño es maximizar la proporción de lógica que puede tratarse como L3, no porque L4 esté excluido, sino porque cada regla que se logra expresar de forma declarativa se vuelve verificable por construcción y deja de consumir revisión humana. La taxonomía opera así como gradiente de esfuerzo, no como frontera de alcance.

4 Riesgos residuales

Los siguientes riesgos no se eliminan con la arquitectura descrita y deben vigilarse de forma explícita.

R-01. Oráculo incompleto. Unas pruebas en verde solo demuestran ausencia de los fallos que ellas exploran. Un criterio de aceptación incompleto produce un oráculo débil y, con él, la ilusión de corrección. Mitigación parcial: exigir cobertura de particiones y valores límite mediante las técnicas de Int [4], y revisar el oráculo con independencia de la implementación.

R-02. Sesgo de confirmación en la revisión. Revisar código que ya pasa las pruebas induce aprobación superficial. Mitigación parcial: muestreo obligatorio de M-07 e inyección periódica de artefactos defectuosos conocidos para medir la eficacia real de la revisión.

R-03. Deriva del modelo generador. Un cambio de versión del modelo altera el comportamiento del generador sin que cambie ninguna entrada. Mitigación: fijar la versión del modelo como elemento de configuración sujeto a TRC-15, y reejecutar el conjunto de regresión de generación ante cualquier cambio.

R-04. Brecha entre corrección aparente y corrección sustantiva. Las mediciones públicas disponibles muestran que la corrección sintáctica del código generado es muy alta mientras la calidad sustantiva es sensiblemente menor [6]. La arquitectura responde a esto con M-03, pero el riesgo persiste allí donde el oráculo no alcanza.

R-05. Complejidad esencial no eliminable. La distinción de Brooks [7] entre complejidad esencial y accidental sigue vigente: el esfuerzo de precisar qué debe hacer el sistema no desaparece, se traslada a la redacción de requisitos y criterios de aceptación. El proyecto debe planificar ese esfuerzo de forma explícita, porque es donde se concentrará el trabajo humano real.

5 Cambios aplicados a SLCP-DOC-001

Los siguientes cambios se han incorporado a la versión 1.1 de SLCP-DOC-001, emitida junto a este registro.

1. Marco normativo ampliado con Int [2], Int [3] y Int [4].
2. Vocabulario canónico ampliado con `BusinessRule` (prefijo `RUL`) y `TestOracle` (prefijo `ORC`).
3. TRC-23 reescrito: de exigencia de reproducibilidad a exigencia de declaración del régimen de determinismo por artefacto.
4. Nuevos requisitos TGT-08 a TGT-13, que recogen los mecanismos M-01 a M-08.
5. Tabla de formatos de intercambio ampliada con DMN.
6. Criterios de piso ampliados con la condición de oráculo congelado, verde y revisado.
7. Asuntos abiertos actualizados: la estrategia de reconciliación queda resuelta por M-05 y se cierra.

Referencias

- [1] *ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system*. International Organization for Standardization, Geneva, Switzerland, 2023. <https://www.iso.org/standard/81230.html>.
- [2] *ISO/IEC 25059:2023 Software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model for AI systems*. International Organization for Standardization, Geneva, Switzerland, 2023. Extensión específica del modelo de ISO/IEC 25010; ISO anuncia su reemplazo por ISO/IEC DIS 25059. <https://www.iso.org/standard/80655.html>.
- [3] *ISO/IEC TR 29119-11:2020 Software and systems engineering — Software testing — Part 11: Guidelines on the testing of AI-based systems*. International Organization for Standardization, Geneva, Switzerland, 2020. Informe técnico, no norma. <https://www.iso.org/standard/79016.html>.
- [4] *ISO/IEC/IEEE 29119-4:2021 Software and systems engineering — Software testing — Part 4: Test techniques*. International Organization for Standardization, Geneva, Switzerland, 2021. Edición 2; cancela y reemplaza a ISO/IEC/IEEE 29119-4:2015. <https://www.iso.org/standard/79430.html>.
- [5] *ISO/IEC 5055:2021 Information technology — Software measurement — Software quality measurement — Automated source code quality measures*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/80623.html>.
- [6] Veracode. Spring 2026 GenAI code security update: Despite claims, AI models are still failing security, 2026. Informe de la industria; tasa de corrección sintáctica superior al 95% frente a tasa de aprobación de seguridad cercana al 55%. <https://www.veracode.com/blog/spring-2026-genai-code-security/>.
- [7] Frederick P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. ISSN 0018-9162. doi: 10.1109/MC.1987.1663532.